Protein Variant-Effect ML Project Report

Project: Protein Variant-Effect ML Practice Project

Goal:
Build a medical-biology-style machine learning workflow that predicts whether a single amino-acid substitution is stabilizing, then extend that workflow to continuous stability-effect prediction using ddG_ML_float.

Dataset and scope:
- Source dataset: FECA_ECOLI_Tsuboyama_2023_2D1U.csv from substitutions_raw_DMS.zip
- Modeling subset: single mutants only
- Classification target: Stabilizing_mut
- Regression target: ddG_ML_float

Why this project matters:
This project mirrors the structure of many molecular-biology and medical-bioinformatics pipelines: variant preprocessing, biologically motivated feature engineering, train/validation/test separation, model comparison, model interpretation, and careful documentation of limitations.

Workflow Summary

Workflow steps taken:
1. Located the target CSV inside the ZIP archive and inspected its columns and sample rows.
2. Filtered the data to single mutants only to create a clean first-pass variant-effect problem.
3. Parsed mutation strings like Q1E into wild-type residue, position, and mutant residue.
4. Built amino-acid chemistry features including hydrophobicity, charge, volume, polarity, aromaticity, and conservative-change flags.
5. Added local sequence-window features around each mutation site to capture context-dependent effects.
6. Split the dataset once into train, validation, and test subsets and reused that split for all models.
7. Trained a baseline classifier and progressively stronger tabular models.
8. Added boosted-tree models with XGBoost and LightGBM once the local environment supported them.
9. Added a stacking meta-learner alongside the existing models rather than replacing them.
10. Extended the project to regression for ddG_ML_float and repeated the same model-comparison pattern.
11. Added interpretation layers using logistic coefficients, permutation importance, and SHAP for tree-based models.
12. Produced reusable scripts, notebooks, figures, metrics files, README documentation, and this final report.

Results: Classification

Classification validation results:
- majority_baseline: F1=0.0000, ROC-AUC=0.5000, PR-AUC=0.0205
- logistic_regression: F1=0.7273, ROC-AUC=0.9961, PR-AUC=0.8929
- random_forest: F1=0.4000, ROC-AUC=0.9607, PR-AUC=0.4762
- xgboost: F1=0.4000, ROC-AUC=0.9581, PR-AUC=0.7778
- lightgbm: F1=0.8571, ROC-AUC=0.9699, PR-AUC=0.7870
- stacking_classifier: F1=0.5333, ROC-AUC=0.9948, PR-AUC=0.8750

Selected classifier: lightgbm
Held-out test result for selected classifier: F1=0.4000, precision=1.0000, recall=0.2500, ROC-AUC=0.8259, PR-AUC=0.6116

Interpretation: LightGBM produced the strongest validation tradeoff under the project's model-selection rule. The stacking classifier was competitive in ranking metrics, but it did not surpass LightGBM on validation F1, so it was kept as an additional model rather than replacing the existing best classifier.

Results: Regression

Regression validation results:
- dummy_regressor: MAE=0.6427, RMSE=0.7932
- random_forest_regressor: MAE=0.3591, RMSE=0.5249
- xgboost_regressor: MAE=0.3675, RMSE=0.5443
- lightgbm_regressor: MAE=0.3675, RMSE=0.4897
- stacking_regressor: MAE=0.3503, RMSE=0.5034

Selected regressor: stacking_regressor
Held-out test result for selected regressor: MAE=0.3478, RMSE=0.4854

Interpretation: The stacking regressor slightly improved on the best individual regressors, which suggests that different base learners captured complementary signals for continuous stability prediction.

Problems Encountered and Solutions

Problems encountered and how they were solved:
- ZIP loading issue: Pandas ZIP-path loading failed in this Windows environment. Solution: switched to Python's zipfile module and loaded the CSV through a text wrapper.
- Label parsing issue: the dataset used '-' placeholders and string-valued labels. Solution: normalized placeholders to missing values, converted numeric targets explicitly, and mapped true/false strings into binary labels.
- Parallel worker permission issue: tree models using multi-process helpers raised Windows access errors. Solution: forced single-process execution for forests and permutation importance so the workflow remained portable.
- Markdown export dependency issue: Pandas to_markdown required tabulate, which was not installed. Solution: replaced it with a small built-in markdown table formatter.
- Package availability issue: xgboost, lightgbm, and shap were not initially available. Solution: installed them and integrated them into the existing pipeline.
- SHAP compatibility issue: LightGBM and sparse transformed matrices produced shape mismatches. Solution: densified the transformed matrix for SHAP and normalized SHAP outputs across model types.
- Documentation drift issue: after model upgrades and stacking, the README files no longer matched the code. Solution: updated the README files to reflect the current model lineup, SHAP usage, and best-model outcomes.

Modeling Rationale

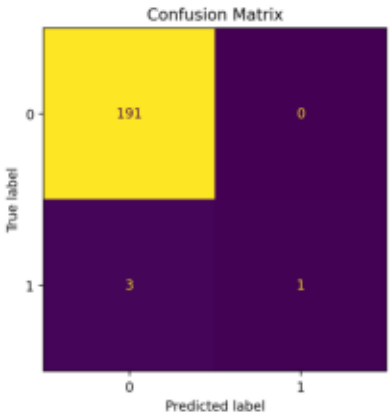
Modeling choices and why they were used:
- majority_baseline: included to show what performance looks like without real learning. This protects against overclaiming success on an imbalanced label.
- logistic_regression: a strong, interpretable baseline for structured biological features. It is useful for understanding which substitutions and contexts are linearly associated with stabilizing outcomes.
- random_forest: a nonlinear baseline that handles tabular interactions well without extensive tuning. It is robust and easy to compare against boosted-tree methods.
- xgboost: added as a stronger gradient-boosting option. It often performs well on structured/tabular data where feature interactions matter.
- lightgbm: added because it is another strong tree-boosting method, often efficient and competitive on medium-sized tabular problems. In this project it became the best classifier under the validation rule.
- stacking_classifier: added as a meta-learner alongside the individual classifiers. Its role is to combine predictions from the existing base models rather than replace them. It was competitive but not the final classifier winner on the validation rule.
- dummy_regressor: included as a no-skill baseline for effect-size prediction.
- random_forest_regressor, xgboost_regressor, lightgbm_regressor: used to compare different nonlinear approaches for continuous stability prediction.
- stacking_regressor: added as a meta-learner for regression. In this project it became the best regressor, suggesting the individual regressors captured complementary predictive signals.

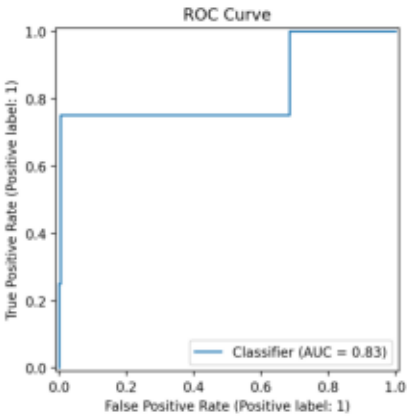
Interpretation choices and why:
- Logistic coefficients provide a simple, high-level biological read on which encoded features push predictions up or down.
- Permutation importance gives a model-agnostic view of which inputs matter for predictive performance.
- SHAP was used for tree-based interpretation because it provides locally grounded feature-attribution summaries and works well for boosted trees.

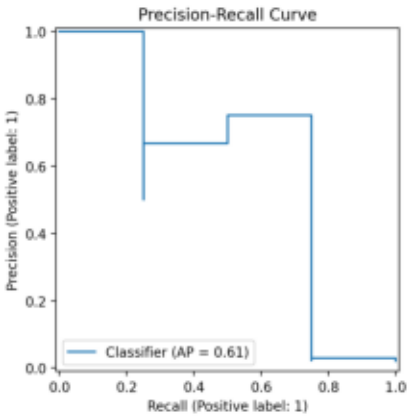
Conclusions

Project conclusions:
- Best classifier: lightgbm
- Best regressor: stacking_regressor
- The project demonstrates that biologically motivated tabular feature engineering is enough to produce useful variant-effect models on this dataset.
- LightGBM achieved the strongest classification validation score under the chosen selection rule.
- The stacking regressor slightly outperformed the individual regressors, showing that ensemble meta-learning can help for continuous stability prediction even when it is not the best classifier.
- This remains an experimental protein-stability project, not a clinical diagnostic model, but it is a strong practice workflow for medical biology and bioinformatics.

Visual: Classification Summary

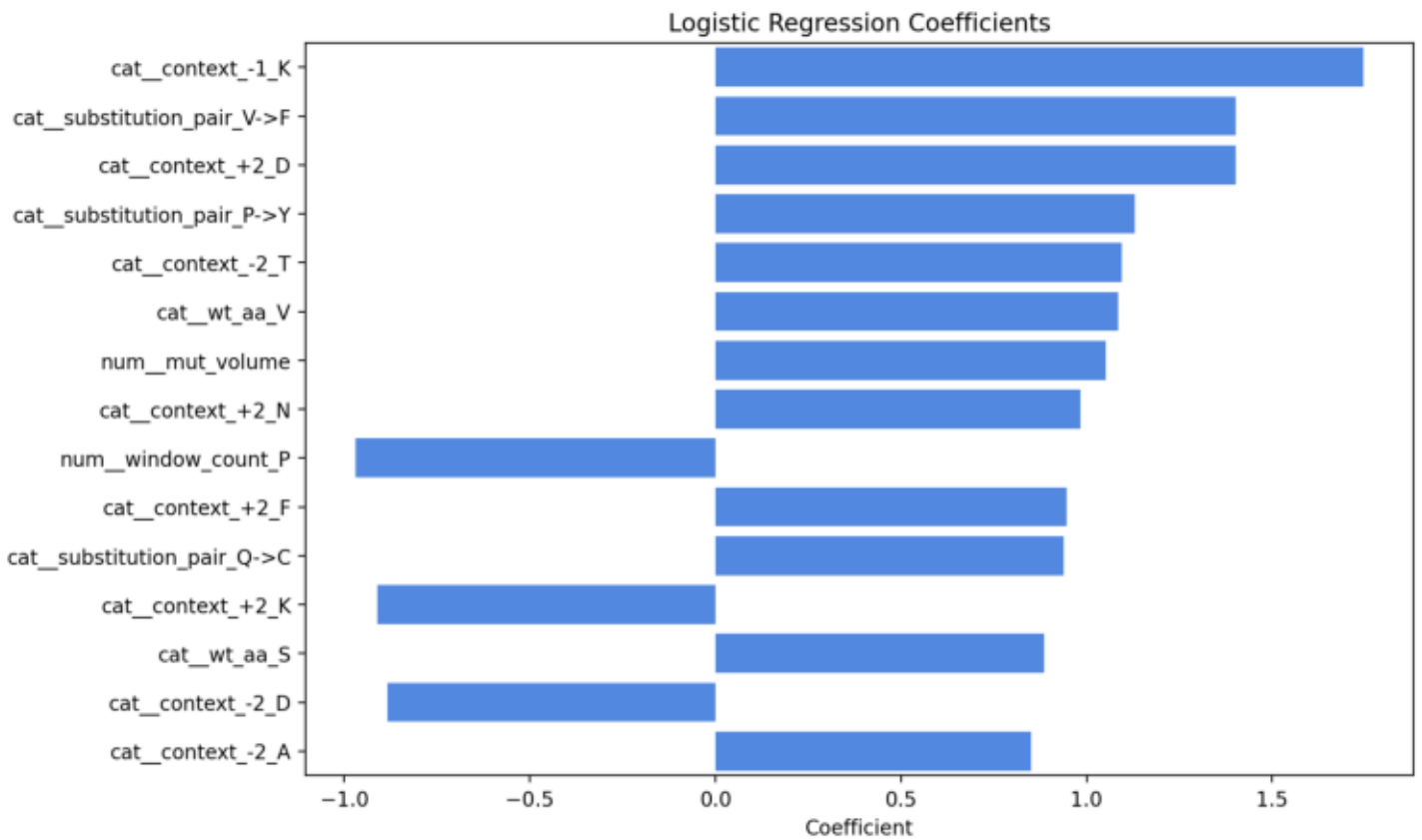






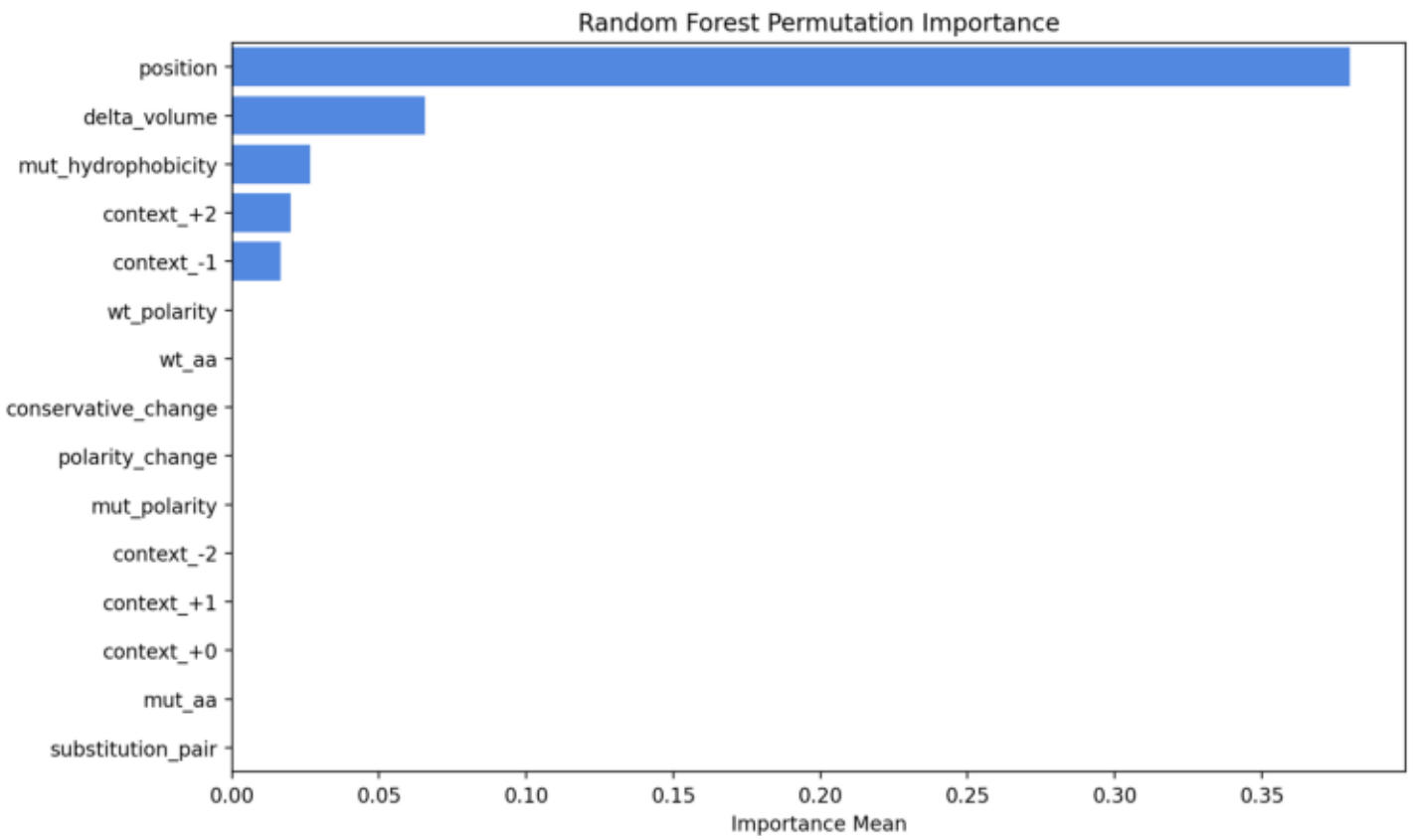
This figure shows the confusion matrix, ROC curve, and precision-recall curve for the selected classifier on the held-out test set.

Visual: Logistic Feature Importance



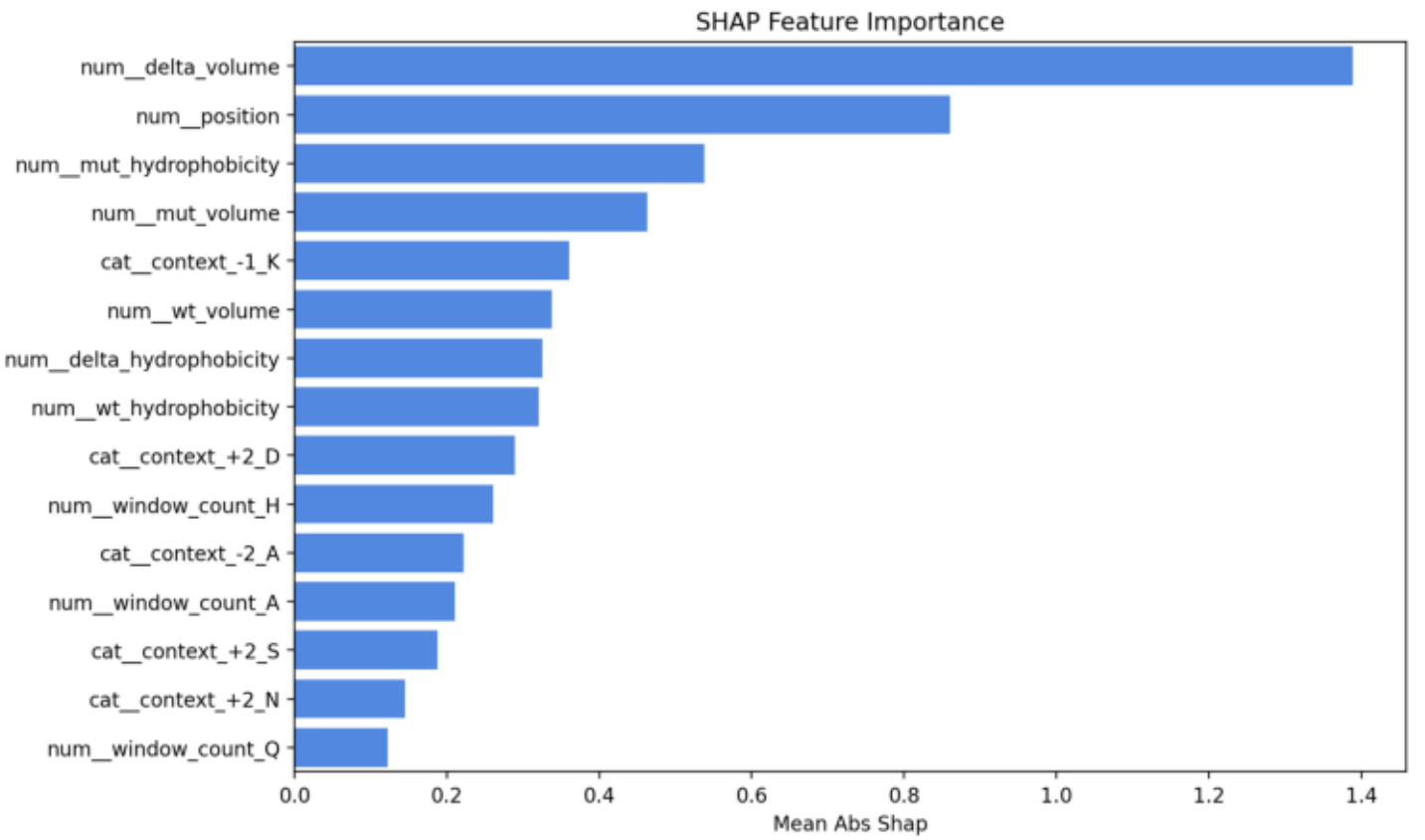
These coefficients show which encoded mutation and context features have the strongest linear association with the stabilizing label.

Visual: Permutation Importance



Permutation importance highlights which raw feature groups most affect predictive performance when their values are shuffled.

Visual: SHAP Feature Importance



SHAP values summarize which tree-model features contributed most strongly to predictions across the evaluation set.